Cohort: technical design

Android · iOS · Web · 23 October 2026

The technical blueprint for building Cohort for Android, iOS and the web from one codebase, in one language the founder already knows: TypeScript, with React Native and React Native Web. This document covers the platform choice and why, the app and backend architecture, the data model, AI integration, notifications, security and compliance, and deployment.

1. The platform decision

The requirement: one familiar language across Android, iOS and web, so multiplatform development stays simple and the same person can work on all three. Two realistic candidates: React Native with React Native Web (TypeScript everywhere), or Flutter (Dart everywhere).

One language across all three platforms — React Native

Both qualify, but TypeScript also covers the backend, so it is one language end to end. Dart usually pairs with a second language for the backend.

Familiarity for the founder — React Native

JavaScript and React are already known from earlier projects. Dart would be a new language to learn first.

Web version quality — React Native

React Native Web renders real DOM, so the web app feels native to the browser with normal SEO and accessibility. Flutter Web paints to a canvas and is heavier.

Hiring and community in India — React Native

A very large React and React Native talent pool versus a smaller, growing Flutter one.

Ecosystem for this app — React Native

Expo covers push, auth, over-the-air updates and store builds out of the box. Flutter needs more manual wiring.

UI polish and animation — Flutter

Flutter's engine is excellent here. React Native's Reanimated and Skia still cover Cohort's card flip and sheet animations comfortably.

AI and backend SDKs — React Native

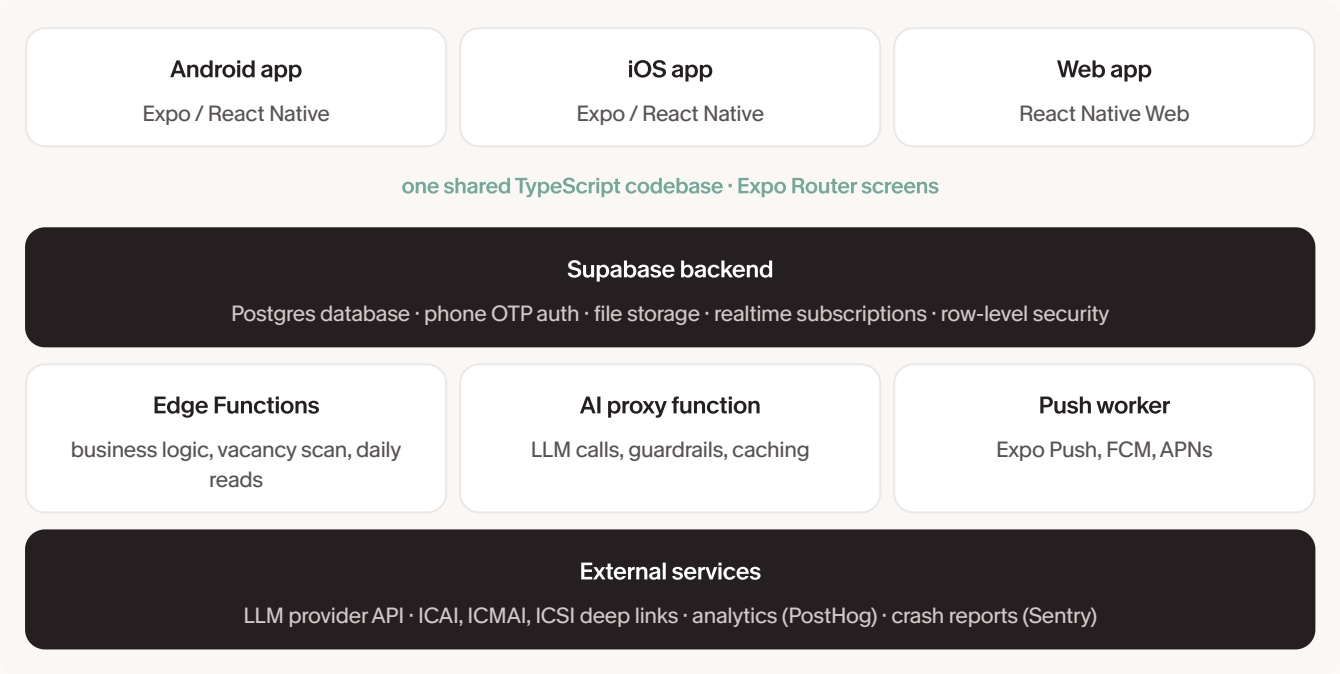
First-class TypeScript SDKs for every LLM provider; Dart mostly glues REST calls by hand.

> **Recommendation**
>
> React Native with Expo and TypeScript, plus React Native Web for the web version. One language end to end (app, web, backend functions), the founder already thinks in JavaScript and React, hiring is easy, and Expo removes most of the mobile plumbing. Flutter is the better animation engine, but Cohort\'s UI is cards, lists and sheets: React Native handles it comfortably, and the language familiarity wins.

Language	TypeScript everywhere: mobile, web, backend
Mobile	React Native with Expo (managed workflow)
Web	React Native Web via Expo Web, same components
Backend	Supabase (Postgres, Auth, Storage, Realtime, Edge Functions in TypeScript)

AI	LLM API through a backend proxy, never directly from the app
Push	Expo Push, which routes to FCM (Android) and APNs (iOS); web push for the browser

2. Architecture overview



Speed to a real launch

Why Supabase for the backend

Supabase gives Postgres, phone-OTP authentication, file storage for resources, realtime subscriptions for chat, and row-level security policies, all on day one, with Edge Functions written in the same TypeScript. The alternative, a custom Node.js (NestJS) API plus Postgres, offers more control but adds weeks before the first screen works. Start on Supabase; the schema and security policies are plain Postgres, so migrating later is possible if scale demands it.

3. The app: one codebase, three platforms

1. **Navigation**
   Expo Router mirrors the prototype exactly: a bottom tab group with Feed, Resources, Community, Work and ICAI (Community centered), Messages as a stack screen from the header icon, and the create sheet as a modal route over any screen.

2. **Shared UI kit**
   A component library (cards, chips, sheets, avatars, role pills) built once on React Native primitives and the warm terracotta design tokens from the prototype. React Native Web renders the same components as DOM for the browser.

3. **State and data**
   TanStack Query for server data with caching and offline snapshots; Zustand for local UI state (sheet open, filters, FAB visibility on scroll).

4. **Animations**
   React Native Reanimated for the daily-reads card flip, the bottom sheet, and the auto-hiding + button; the same code runs on all three platforms.

5. **Platform differences**
   Kept behind small adapters: notifications, file downloads, sharing. Everything else is identical across platforms.

4. Data model

Core tables in Postgres. Row-level security policies enforce who can read and write what (a student sees only their own applications; a firm sees only its own openings).

users, profiles

Auth identity; profile with course, level, attempt, city, role (student, mentor, firm, moderator).

verifications

Membership numbers and documents for mentor and firm badges, with review status.

posts, comments, likes

Feed content with type (post, announcement, broadcast) and pinning.

daily_reads, daily_read_items

The curated Top 10 per day per audience, AI-drafted then human-reviewed.

resources

Files with course, level, group, subject, type, author, download counts.

groups, group_members

Community groups, purpose, admin, membership.

jobs, applications

Openings with category, pay, tags, firm; applications with status (applied, viewed, shortlisted).

threads, messages

One-to-one conversations with unread counts.

mcq_sets, mcq_questions, mcq_attempts

Practice sets by course and subject, with per-user attempt history and scores.

ai_sessions, ai_messages

Cohort AI conversations, for context continuity and cost tracking.

notifications

Per-user notification records and delivery state.

reports

Moderation reports with review queue states.

5. AI integration

Cohort AI (summaries, concept clarity, revision plans) and MCQ generation run through one Edge Function that proxies to the LLM provider. The app never talks to the LLM directly: the proxy holds the keys, applies the

system prompt, enforces guardrails and rate limits, and logs cost per user.

- Study scope: the system prompt restricts answers to CA, CMA and CS study topics, exam technique and careers; anything else gets a polite redirect.
- Grounding: answers prefer ICAI, ICMAI and ICSI study material and bare acts; a retrieval layer over uploaded modules can be added in Phase 3 without changing the app.
- MCQ generation: a scheduled job drafts question sets per subject; a human reviewer approves before they reach students.
- Daily reads: the Top 10 is drafted by AI from the day's announcements and updates, then approved by an editor before publishing each morning.
- Cost control: per-user daily limits, cached answers for common asks, and a monthly budget ceiling with alerts.
- Privacy: chats are not used to train anything; retention follows the privacy policy with deletion on request.

6. Notifications

Morning, 7 AM local
Today's Top 10 reads are ready.

New message
Sender name and preview; opens the thread.

Vacancy scan match
Count of new matching openings; opens Work filtered.

Application status
Viewed, shortlisted, rejected; from the firm.

Official announcement
Pinned BoS or institute announcements for the user's course.

Group activity (digest)
A quiet daily digest, off by default for noisy groups.

Delivery: Expo Push API from a backend worker, routing to FCM on Android, APNs on iOS and web push in the browser. Every type has a per-user setting.

7. Auth, security and compliance

- Sign-in: phone number with OTP (Supabase Auth with an Indian SMS provider). No passwords to forget for a student audience.
- Sessions: secure refresh tokens stored in Expo SecureStore on device; httpOnly cookies on web.
- Authorization: Postgres row-level security on every table; roles (student, mentor, firm, moderator) gate writes such as posting openings or pinning announcements.
- Verification: mentors and firms upload proof; moderators approve in an admin console before badges appear.
- Data protection: TLS in transit, encryption at rest, signed URLs for private files, DPDP Act 2023 duties (consent notice, export and deletion on request, grievance contact).
- Abuse: rate limits on posting and messaging, report and block flows, a moderator review queue.

8. Build, release and operations

1. **Environments**
   Dev, staging and production Supabase projects; feature flags for unfinished modules.
2. **CI**
   GitHub Actions: typecheck, lint and tests on every pull request; preview builds on merge.
3. **Mobile release**
   EAS Build and Submit for Play Store and App Store; EAS Update ships JS fixes over the air without a store review.
4. **Web release**
   The same codebase deployed as a static web app (Vercel or Netlify) on the product domain.
5. **Observability**
   PostHog for product analytics and funnels; Sentry for crash and error reporting across all three platforms.
6. **Design source of truth**
   The clickable prototype: tokens (terracotta on cream, rounded cards) become the shared theme file; screens match it one to one.

9. Phasing, mapped to the roadmap

1. Phase 1 (weeks 1-8): Expo app shell with the five tabs, Supabase auth and profiles, Feed with daily reads, Resources with storage, 1:1 messaging over Realtime. Team: 1-2 React Native developers.
2. Phase 2 (weeks 9-14): Jobs and applications, groups, connections, verification console for moderators.
3. Phase 3 (weeks 15-20): ICAI services grid, Cohort AI proxy and chat, MCQ engine and history, vacancy scan worker, push notifications.
4. Phase 4 (weeks 21+): analytics-driven iteration, monetization plumbing (featured listings first), ICMAI and ICSI service grids.

> **A pragmatic shortcut**
>
> If speed matters more than polish for Phase 1, the same TypeScript and React skills carry over to a FlutterFlow or Lovable-style low-code MVP. The schema in this document works either way, so nothing designed here is wasted.

Written 23 Oct 2026 alongside the PRD and the app-flow document. The platform pick is a recommendation; nothing is built or purchased yet.